

AutoForge: A Self-Evolving Multi-Agent System for Autonomous Tool Creation, Retrieval, and Distribution via MCP

COMP7608 Large Language Models | Group Project Proposal

1. Group Members

Name	Student ID
Liu Xinquan	3036659637
Cheung Yu Lung	3035477616
Lyu Linze	3036658619

2. Project Goal

We propose **AutoForge**, a self-evolving multi-agent system that shifts the paradigm from *"providing tools to agents"* to *"agents forging and accumulating their own tools."* AutoForge autonomously writes, validates, stores, and reuses Python tools. Once a tool is forged for a task type, semantically similar future requests bypass code generation entirely — they retrieve and invoke the cached tool directly via a semantic search layer (Tool-RAG).

Core Objective: Build a system where an LLM-based agent can autonomously extend its own capabilities at runtime, reducing per-query latency and token cost for recurring task families by replacing redundant code generation with efficient tool retrieval and reuse.

3. Motivation and Challenges

3.1 Motivation

As highlighted in course lectures, LLMs face the **"Island Dilemma"** — they are disconnected from the dynamic world and rely on external tools to act upon it. While tool-augmented agents partially address this, two deep-rooted bottlenecks persist in current systems:

- **Pre-defined Tool Dependency:** Agents operate on a fixed, manually curated toolset. If a required capability is absent, the agent fails — it cannot autonomously fill the gap.
- **Redundant Code Generation in Code Agents:** Code Agents (e.g., Code Interpreter) can write ad hoc scripts to solve novel problems, but regenerate code from scratch on every request. This leads to high per-query latency and token cost, with no knowledge accumulated across sessions.

AutoForge addresses both bottlenecks simultaneously: agents can create new tools on demand *and* accumulate them for future reuse, turning one-off solutions into a growing, shared capability library.

3.2 Challenges

- **Tool Generalization:** Generated tools must be sufficiently general to serve paraphrased or semantically related future queries, not just the specific query that triggered their creation.
- **Semantic Retrieval Accuracy:** A false-positive tool retrieval (matching the wrong tool) causes silent incorrect outputs, which is harder to debug than an outright failure.
- **Safe Code Execution:** Autonomously generated and executed code poses sandbox escape risks if not properly isolated.
- **Dynamic Tool Registration:** Registering new API endpoints at runtime without destabilizing the running server requires careful versioning and schema management.
- **Evaluation Complexity:** Measuring both the correctness of tool outputs *and* the efficiency gains from reuse requires a multi-dimensional evaluation framework that standard benchmarks do not provide out of the box.

4. Potential Solutions

AutoForge is built on **Python**, **FastAPI**, **LangGraph**, **ChromaDB**, and **Sentence-Transformers** (all-MiniLM-L6-v2 for embedding). The system is structured into four sequential phases. The system is structured into four sequential phases, as illustrated in Figure 1 below.

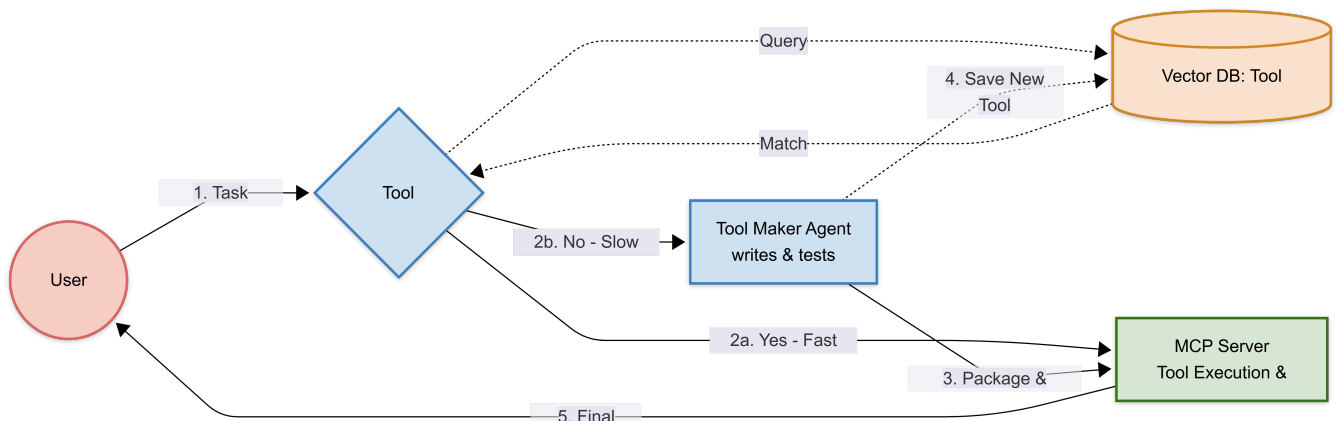


Figure 1. AutoForge system architecture: Intent Routing and Tool-RAG pipeline.

Phase 1 — Intent Routing & Tool-RAG

Incoming user queries are encoded into dense vector embeddings using a pre-trained Sentence-Transformer model. These embeddings are matched against stored tool descriptions in ChromaDB (HNSW-based approximate nearest-neighbor search, achieving sub-linear $O(\log n)$ retrieval time):

- **Fast Path (similarity $\geq \tau$):** Structured parameters are extracted from the query via a lightweight LLM call and dispatched directly to the matched tool. No code generation occurs.
- **Slow Path (similarity $< \tau$):** The query is routed to the Tool Maker Agent pipeline for new tool creation.

Phase 2 — Multi-Agent Tool Forging (ReAct Loop)

When no suitable tool exists, a LangGraph-orchestrated ReAct pipeline generates one. Three sub-agents collaborate in a controlled loop:

- **Planner Agent:** Decomposes the natural language request into an ordered sequence of programmatic steps and specifies expected input/output types.
- **Coder Agent:** Generates a self-contained Python function matching the plan, with type hints and a descriptive docstring (required for AST packaging downstream).
- **Verifier Agent (Sandbox):** Executes the generated code in an isolated subprocess environment with a configurable timeout. If execution fails, the full traceback is fed back to the Coder Agent for iterative refinement (hard cap: N = 5 retries).

Phase 3 — AST Parsing & Tool Packaging

Once code passes verification, the **Packager Agent** uses Python's `ast` module to statically extract the function name, parameter names, type annotations, and docstring. It automatically generates an OpenAI-compatible JSON Schema for the tool, stored alongside the function code. The docstring embedding is inserted into ChromaDB as the semantic retrieval key for future queries.

Phase 4 — MCP Integration & FastAPI Distribution

Successfully packaged tools are dynamically registered as endpoints in a **FastAPI** backend using an `APIRouter`-based dynamic registration pattern. The backend is wrapped with the **Model Context Protocol (MCP)**, transforming AutoForge into a universal "Tool Server." This enables external MCP clients (e.g., Claude Desktop, Cursor IDE) to discover and invoke AutoForge-generated tools without any manual configuration.

4.1 Risk Mitigations

Risk	Impact	Mitigation Strategy
Sandbox Escape / Unsafe Code	High	Isolate via Docker or E2B; restrict network, filesystem, and time resources.
False-Positive Tool Retrieval	Medium	Two-stage check: embedding similarity + LLM compatibility verification before fast-path dispatch.
Tool Overfitting (non-generic code)	Medium	Explicit generalization constraints in Coder Agent prompt; secondary docstring refinement pass.
Infinite Verification Loops	Low	Hard cap at N = 5 retries; return structured failure after limit.
Dynamic Route Registration	Medium	Tool-versioned endpoints; rebuild OpenAPI schema on registration events.

4.2 Technology Stack

Component	Technology / Library
Multi-Agent Orchestration	LangGraph (ReAct graph), LangChain (LLM wrappers)
LLM Backend	GPT-4o / Claude-3.5-Sonnet via API (configurable)
Embedding Model	<code>sentence-transformers/all-MiniLM-L6-v2</code> (local)
Vector Database	ChromaDB (HNSW index, cosine similarity)
Code Execution Sandbox	Docker container (isolated) or E2B cloud sandbox
AST Parsing	Python built-in <code>ast</code> module
API Backend	FastAPI with dynamic <code>APIRouter</code> registration
Tool Distribution Protocol	Model Context Protocol (MCP)
Frontend	Streamlit

5. Experimental Dataset Description & Evaluation Plan

5.1 Datasets

We adopt a hybrid dataset strategy combining a custom-curated benchmark with established public datasets.

Primary Dataset — "OS & Office Automation" Benchmark (Custom)

We will curate 60–100 practical automation queries stratified across four difficulty levels. Each sample follows the structure: `{"query": "...", "expected_output": "...", "difficulty": 1}`. Crucially, we include **3–5 semantically equivalent paraphrases per task** to evaluate retrieval robustness — a dimension absent from most existing tool benchmarks.

Level	Task Category	Example Query
L1 — Basic	String / Math Ops	"Count the vowels in a given text string."
L2 — File I/O	CSV / JSON Processing	"Read a CSV and multiply the 'price' column by 1.2."
L3 — External API	HTTP / Web Requests	"Fetch the current BTC/USD price from a public API."
L4 — Multi-step	Chained Operations	"Download a CSV from a URL, clean nulls, return summary stats."

Supplementary Public Datasets

Dataset	Size	Purpose
HumanEval (subset)	50 problems	Benchmark raw code-generation capability of the Coder Agent in isolation, establishing a quality baseline.
ToolBench (subset)	30 tools	Evaluate whether AutoForge-generated tool schemas conform to standard API invocation conventions.

5.2 Evaluation Plan

We evaluate AutoForge across three orthogonal dimensions:

Correctness — Task Success Rate

Tool outputs are compared against ground truth using exact match for deterministic tasks (string manipulation, CSV transforms) and fuzzy/tolerance-based match for numeric API results. Partial credit is reported for tasks where the tool executes successfully but produces an off-by-epsilon result, distinguishing execution failures from logical errors.

Efficiency — Latency Reduction

Wall-clock execution time is measured across two conditions:

Condition	Measurement
Cold (Tool Forging)	Full pipeline: LLM plan → code → sandbox verify → package → execute
Warm (Tool Retrieval)	ChromaDB retrieval → parameter extraction → direct execution
Speedup Metric	Speedup = Cold Latency / Warm Latency (hypothesis: ≥ 5–10×)

Results are reported as mean ± std across all L1–L3 tasks.

Generalization — Tool Reuse Rate (TRR)

$$TRR = \frac{Q_{\text{fast}}}{Q_{\text{total}}}$$

where:

- Q_{fast} denotes the number of queries served via the fast path,
- Q_{total} denotes the total number of queries.

TRR is measured after all unique task types have been processed.

A high TRR indicates that the system successfully abstracts reusable tools.

Target: $TRR \geq 0.6$ on the paraphrase test set.

Evaluation Protocol

1. **Phase 1:** Process all unique queries (cold path only) → measure Task Success Rate and Forging Latency.
2. **Phase 2:** Re-run the same queries and paraphrases (warm path) → measure Retrieval Latency and TRR.
3. **Phase 3:** Coder Agent standalone on HumanEval subset → measure pass@1 code correctness.
4. **Ablation:** Disable RAG (always forge) vs. full system → quantify the cache benefit.

6. Workload Distribution

Member	Role	Responsibilities
Liu Xinquan	Agent Logic & Algorithms	ReAct multi-agent loop (LangGraph); prompt engineering for Planner, Coder, and Verifier agents; sandbox execution environment (Docker/E2B); iterative refinement with retry caps.
Cheung Yu Lung	Backend, RAG & MCP	FastAPI dynamic route registration; ChromaDB integration and threshold tuning; AST parsing pipeline and JSON Schema generation; MCP Server implementation and external client testing.
Lyu Linze	Frontend, Data & Evaluation	Streamlit UI; benchmark curation (60–100 queries + paraphrases); automated evaluation scripts for correctness, latency, and TRR; ablation study design and results visualization.

7. Proposed Timeline

Week	Milestone
Week 1–2	Environment setup; LangGraph agent scaffold; Coder/Verifier loop (no RAG)
Week 2–3	ChromaDB integration; embedding pipeline; threshold calibration
Week 3–4	AST Packager; JSON Schema generation; tool storage pipeline end-to-end
Week 4–5	FastAPI dynamic registration; MCP server; external client test
Week 5–6	Benchmark curation (60–100 queries + paraphrases); evaluation scripts
Week 6–7	Full system integration; ablation runs; latency profiling
Week 7–8	Streamlit UI; final report writing; results visualization